VeriBrand Round 2 Update

Team 27 | DHEERAJ M G and Charan Teja Arangi | 22 October 2026

Summary

VeriBrand is a working text-first brand-governance demo. Creators review content against approved fictional Acme Hydrate knowledge, inspect cited findings and make explicit decisions. New versions pass to a Manager, with Reviewer sign-off for retained medical risks. The live core workflow and an uncached Groq request passed the submission checks.

Live demo: https://veribrand.onrender.com

Repository: https://github.com/DheerajMG-Bng/veribrand

Roles are simulated, not authentication. This shared hackathon deployment is not a production service. The team observed three informal volunteer sessions after deployment; their notes and the supplied team contributions are included.

Round 1 plan and delivered implementation

This comparison describes our Round 1 plan (Team 27) and the Round 2 delivery.

Our Round 1 plan	Delivered in Round 2	Why it changed
Gemini API	Groq via HTTPX with configured fallback	Use the existing provider; degrade safely
Supabase/PostgreSQL	SQLAlchemy 2 and SQLite	Reduce external setup for the demo
pgvector retrieval	Complete compact 19-item KB prompt	Small supplied knowledge set
Separate React hosting	React build served by FastAPI	One Python deployment and one origin
Real user roles	Header-selected demo roles	Finish approval semantics before identity
Image and DOCX/PDF review in scope	Text review only; formats in Round 3	Preserve the complete core workflow
Three planned improvements	Clarity insights, multi-format and evaluation harness retained for Round 3	Keep the planned direction with concrete designs

Additional Round 2 decisions include metadata-driven rules, grounding and replacement revalidation, plus a KB-matched precomputed sample cache to reduce latency and provider use.

Completed features

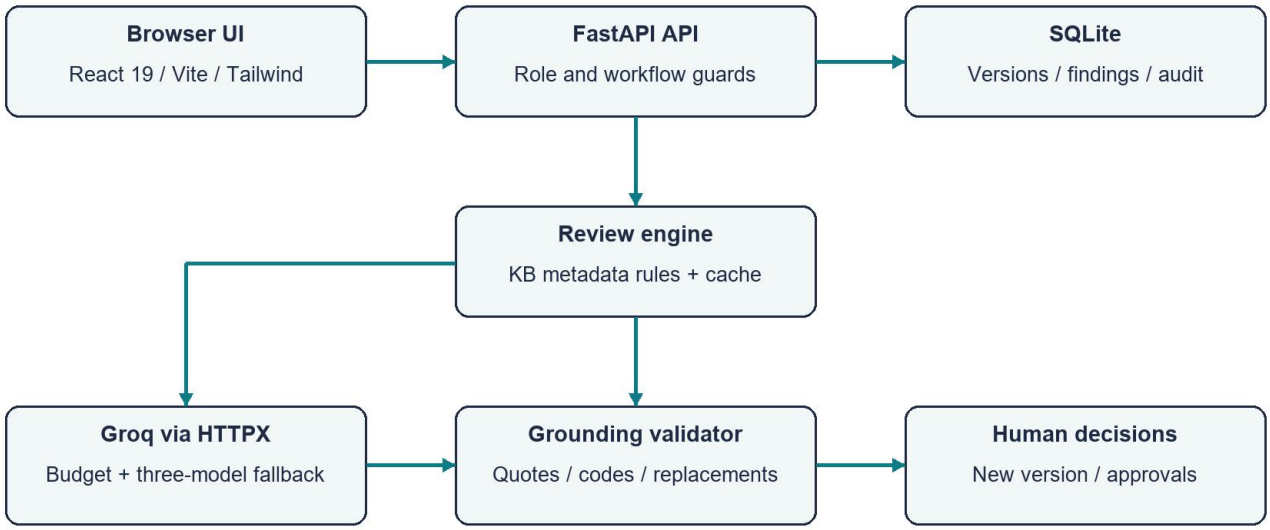
The following were exercised on the public service by HTTP requests or browser use on 22 October 2026. Automated checks are not human usability sessions.

- Guided demo creates sample 1; sample selection loads and saves a draft.
- Rules-only review highlights draft quotes and returns real KB codes; clicking T-02 opens its approved text.
- Live AI review returns an AI-suggested unsupported claim, labelled Unverified, needs a human with Accept disabled.
- Accept, edit and reject save explicit decisions; reject without a reason is rejected.
- Apply decisions creates a new version. Review is required before submitting the latest version, and unapplied fixes block submission.
- Version history, manual revisions and plain before/after comparison work.
- Comments and return notes persist against content.
- Manager approval routes retained medical risks to Reviewer; reasoned sign-off reaches Approved.
- Return with comment, revision, re-review and resubmission work end to end.
- Audit records the tested actions, decisions, role, time and AI involvement.
- Knowledge Base groups 19 items. The live Admin editor saved a title change and restored the original.
- Role switching exposes the relevant controls; wrong-role approval returns 403. AI status displays available after loading.
- The public API accepts 5,000 characters, rejects 5,001 and returns a friendly review-rate 429 with Retry-After.
- FastAPI serves the React app and frontend deep links from one deployment.

Verification scope

Browser checks covered the guided demo, sample form, rules review, cited source modal, AI labels, role selector, status chip and admin editor. Public HTTP checks exercised all decisions and approval states, diff, comments, audit, length boundary and rate limiting. Exact evidence and methods are committed under docs/submission/evidence and docs/submission/VERIFICATION.md.

Architecture and technology



Layer	Implementation
UI	React 19.1.0, React Router 6.30.1, Vite 6.3.5, Tailwind 3.4.17; navy and teal UI
API and validation	Python, FastAPI 0.115.12, Pydantic 2.11.4, Uvicorn 0.34.2
Data	SQLAlchemy 2.0.40 and SQLite; content, versions, findings, comments, audit, review cache and AI usage
Provider	HTTPX 0.28.1 POST to api.groq.com/openai/v1/chat/completions; no provider SDK
Model chain	openai/gpt-oss-120b -> openai/gpt-oss-20b -> qwen/qwen3.8-27b -> rules only
Deployment	One Render Python service; committed static React assets; health route /api/health

API and workflow

The /api routes provide knowledge and samples; content creation and reads; review; finding decisions; fix application and versions/diff; submit, approve, return and reviewer-signoff; comments/audit; AI status; and Admin knowledge/reset actions. X-Role selects the simulated role. State-changing operations write audit entries. Approval is an internal state and does not publish externally.

The primary model was verified live during the submission check; the fallback appeared in the original saved run. The configured third model remains not verified live: one minimal JSON-mode call on 22 October 2026 returned HTTP 403, model_permission_blocked_org, because the organization blocks it. No retry or app-code change was made.

Technical decisions and difficult parts

Grounding and human control

Rules read KB metadata rather than branching on policy codes. The validator maps quotes to actual draft offsets, removes unmatched quotes, nulls invented sources and restricts unsupported findings. Overlap merging preserves the higher-severity issue, supporting codes and Reviewer risk. It does not prove semantic truth merely because a code exists.

Suggested replacement phrases are rechecked for content violations; whole-draft channel disclosure is checked on mandatory re-review. Accepted and edited spans apply from right to left, so offsets stay valid. Nothing is automatically applied, and unapplied decisions cannot be submitted as though they changed the text.

Provider reliability and caching

One logical AI analysis can make retries/fallback requests. The client uses a 30-second per-request timeout, JSON output, low temperature and a 2,500 completion-token cap. GPT-OSS uses low reasoning effort, retried once without that field after 400. Short Retry-After is honoured; errors fall through to rules-only results.

Per-model minute usage is in memory and daily usage is in SQLite; local skip thresholds are 6,400 minute tokens and 180,000 daily tokens. SHA-256 cache keys include normalized text, channel, KB hash and AI flag. Precomputed samples preserve genuine model/time provenance. These are single-process controls and do not see other clients using the provider key.

Libraries datasets and licences

Component	Declared licence or provenance
React, React Router, Vite, Tailwind, PostCSS, Autoprefixer	MIT
FastAPI, Pydantic, SQLAlchemy, pytest	MIT
Uvicorn, HTTPX, python-dotenv	BSD family; see exact package declarations in inventory
Lucide icons	ISC
Prettier development formatter	MIT
Acme Hydrate knowledge and 11 samples	Fictional data supplied in the team build brief; no outside training dataset or fine-tuning
Hosted Groq models	Provider/model terms apply separately; configured names do not promise availability

Full direct/transitive package names, versions and declared licences are committed in docs/THIRD_PARTY_LICENSES.md. No external website template was copied in this build.

Reliability results and known issues

Observed check	Result	Scope
pytest rerun	44 passed, 1 warning	Offline rules, validator, workflow and provider mocks
Rules sample rerun	17 / 17 expected code occurrences	Our own 11 drafts; clean sample 2 has zero findings
AI sample rerun	17 / 17 expected code occurrences	All entries are cached saved responses, revalidated
Fresh public AI request	0 -> 917 tokens; cached=false	Primary GPT-OSS; AI-origin unverified claim
Public workflow	Passed	Decisions, versions, approval, return, comments and audit
Public boundary checks	Passed	5,000 accepted; 5,001 rejected; friendly 429 observed

Recall figures are measured on our own 11 sample drafts, not general accuracy. They are code-occurrence recall, not independently labelled precision/recall. The original live AI sample run had one false positive on clean sample 2; the general terminology validator correction removed it on replay. Both original and corrected records remain available.

Known limits

- Roles are simulated and data is shared; there is no real authentication or tenant isolation.
- Render Free can cold-start and lose SQLite state, audit history and local counters on restart/redeploy/spin-down. Provider quotas still apply.
- The third model is not verified live: one JSON-mode call returned HTTP 403, model_permission_blocked_org, on 22 October 2026. Other uses of the same key are invisible to local counters.
- Confidence is uncalibrated; quote/code validation cannot establish every AI explanation is correct. High-risk overrides can remain with reasons; medical risks need Reviewer sign-off.
- Images, DOCX/PDF ingestion, clarity analytics and benchmark metrics in the UI are not implemented.
- KB changes do not automatically re-review existing versions or snapshot historical policy text. Diff is plain before/after.
- Initial status can briefly say Rules available before loading; rules-only progress copy still mentions AI. The test suite emits a dependency deprecation warning.
- Usability evidence is limited to three informal volunteer sessions; no formal study or independent accuracy evaluation has been done.

Team contributions and disclosure

Team confirmed contributions

Dheeraj created the GitHub repository, the Groq key and the Render deployment; wrote the Round 1 plan; directed the AI-assisted build with the prompts; and handled documentation and submission.

The code was generated by Codex (GPT-6.1 Sol) under Dheeraj's direction. Codex assisted with tests, verification tooling and submission artifacts. Commit author names are not used to infer who completed a task.

Charan

<< Charan reviewed the review-engine code with Dheeraj, ran the test suite, and took part in the live testing, Backend, coding implementation ideas >>

AI and reuse disclosure

Product AI sends draft text, channel, compact approved knowledge and rule summaries to Groq only when enabled on a suitable cache miss. The UI discloses this transfer and asks users to avoid confidential content. The key stays server-side and .env is ignored and untracked. The application uses fictional team-supplied brand data, open-source libraries and Lucide icons; no external website template or additional training dataset was used.

The verified public request returned an AI-origin unsupported claim and increased primary-model daily usage from 0 to 917 tokens. The live UI showed AI-suggested, Unverified, needs a human and disabled Accept. No accounting/label fix was required. An earlier zero alone was inconclusive because cache hits do not add usage and ephemeral counters may reset.

Evidence and source boundaries

Implementation and automated test evidence are in this repository. The comparison names our Round 1 plan (Team 27). Contributions and informal session notes are the team's supplied facts. Round 3 pages describe proposed designs and future acceptance measures, not completed features. Full records: docs/submission/VERIFICATION.md, docs/AI_DISCLOSURE.md and docs/KNOWN_ISSUES.md.

Informal user testing

Informal, n=3. Run and observed by the team on the live site on 22 October 2026, after deployment. The following are the supplied observer notes.

Tester 1

Approximate total session time: about 4 minutes.

Learned the product from the guided demo, completed the journey, relied on the AI-suggested findings and checked them against the knowledge base, and acted as the Manager role.

Tester 2

Approximate total session time: about 6 minutes.

Used a sample draft and switched roles freely. Said a separate Reviewer role felt unnecessary and preferred approving directly as Manager.

Observer note: This is one tester's reaction to the Reviewer step.

Tester 3

Approximate total session time: about 5 minutes.

Used a sample draft, followed the AI suggestions and checked them against the knowledge base.

Common observation

All three relied on the AI-suggested findings and verified them using the visible source line:

Source: organisation-approved knowledge base · Fictional Acme Hydrate

Limits

Three volunteers, informal, observed by the team, no timing of individual tasks, no formal questionnaire. Not statistically meaningful. The test sessions left content items in the live demo workspace.

Usability evidence is limited to three informal volunteer sessions; no formal study or independent accuracy evaluation has been done.

Round 3 improvement 1

Guideline Clarity Insights

Proposed work for Round 3. No user impact or acceptance result below is claimed measured.

User problem

Managers cannot currently see which guidelines repeatedly cause confusion, revisions or overrides. A single draft view hides recurring policy problems.

Concrete technical design

Snapshot policy revision/text on each finding. Store structured override categories alongside original reasons. Aggregate issue frequency, override rate, repeat edits and decision time by policy revision, channel and date, with denominators. Add a Manager/Admin insights endpoint and dashboard linking every alert to permission-checked source findings. Require minimum sample counts; suggested clarifications remain human proposals.

Milestones

- M1: add policy revision snapshots, override categories and fixture-based aggregation tests.
- M2: deliver counts, rates, date filters and drill-down to real evidence.
- M3: add ambiguity signals and optional grounded summaries; test small-sample and access-control cases.

How success will be measured

Proposed acceptance measures: all displayed aggregates exactly match event fixtures; every alert links to its supporting findings; no alerts below the declared sample threshold. In a future human study, compare task completion time for identifying a recurring guideline problem against the draft-by-draft baseline. No improvement percentage is claimed before measurement.

What we cut first if time is short

Cut AI clustering/summaries first, then advanced cohort filters. Keep policy revisions, counts with denominators, override reasons and evidence drill-down.

Round 3 improvement 2

Multi format review of images and DOCX and PDF

Proposed work for Round 3. No user impact or acceptance result below is claimed measured.

User problem

Creators currently must paste text; they cannot submit an image, brochure or document and inspect a finding at its original page or image region.

Concrete technical design

Add private Asset and ExtractedSpan records with file hash, MIME type, extraction coverage, page/paragraph coordinates and version. Validate file signatures and size/page limits; isolate parsing. Extract DOCX paragraphs/tables with python-docx and PDF text with pypdf/pdfplumber; use OCR for scanned pages. For images, check supplied alt text, measurable contrast, palette and logo presence with OCR/vision assistance. Evaluate the requested qwen/qwen3.8-27b only after confirming provider vision support, otherwise select a supported vision model behind an adapter. Reuse rules, grounding, decisions and approval; show cannot determine when evidence or extraction is inadequate.

Milestones

- M1: implement validated uploads and DOCX/selectable-PDF extraction with source locations and coverage warnings.
- M2: add image/OCR ingestion, alt-text review and measured palette/contrast checks.
- M3: add supported vision/logo checks and scanned-PDF OCR; integrate asset versions and source-region review.

How success will be measured

Proposed acceptance measures: every retained finding maps to an original paragraph, page or image region; fixture tests detect corrupt/oversize files and unreadable pages without claiming complete extraction. Measure extraction coverage and per-check precision/recall on labelled format fixtures, then have a human complete review-to-approval on each supported format. Complex contrast must return cannot determine instead of an invented ratio.

What we cut first if time is short

Cut custom logo detection and broad visual aesthetics first, then scanned-PDF OCR if needed. Keep DOCX/selectable-PDF extraction, basic image/alt-text checks, coverage warnings and source-located findings; do not advertise an unsupported format as complete.

Round 3 improvement 3

Reliability and evaluation harness

Proposed work for Round 3. No user impact or acceptance result below is claimed measured.

User problem

The current 11-draft sample set is too small to establish general accuracy, and accepting a suggestion does not prove that it was correct. Users need clearer evidence of what a review system can reliably detect.

Concrete technical design

Build a versioned human-labelled benchmark with separate development/evaluation splits, uncertain cases and adjudication. Record expected code, span, severity and safety labels. Pin results to code commit, KB/prompt hash, provider/model and timestamp; separate cache replays from new calls. Add finding-level correctness/source/severity/safety feedback distinct from accept/reject. Report per-code/channel precision and recall, span quality, fabricated-citation and unsafe-replacement rates, latency, token cost and fallback behavior. Display benchmark date, counts and uncertainty next to metrics; confidence remains uncalibrated until evaluated.

Milestones

- M1: define labels, annotation guide, split and deterministic CI runner with failure injection.
- M2: store reproducible model runs and finding-level feedback; adjudicate disagreements.
- M3: publish benchmark-scoped metrics in the UI, add confidence-bin analysis and compare model/prompt revisions.

How success will be measured

Proposed acceptance measures: 100% of displayed metrics reproduce from stored labels/results; all provider-failure fixtures complete via a documented fallback or rules-only result; no known fabricated citation or rule-violating replacement passes the safety fixtures. Set quality release thresholds only after a labelled baseline exists and show sample counts and uncertainty. Confidence calibration and independent human evaluation are future measurements, not current claims.

What we cut first if time is short

Cut live model-comparison dashboards, automated calibration and continuous paid evaluations first. Keep a fixed labelled benchmark, reproducible offline runner, failure fixtures, feedback records and honest metric scope.